NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

DUARTE JOSÉ LOURENÇO BELO

Bachelor in Computer Science and Engineering

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

MASTER IN COMPUTER SCIENCE AND ENGINEERING

SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon

Draft: June 28, 2026

NOVA SCHOOL OF SCIENCE & TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

COST AND OPERATIONS OPTIMIZATION IN MULTI-CLOUD ENVIRONMENTS OVER GENAI FRAMEWORKS

DUARTE JOSÉ LOURENÇO BELO

Bachelor in Computer Science and Engineering

Supervisor: Tomás Hipólito
PhD, Closer Consulting

Co-supervisor: Margarida Mamede
Professor, NOVA School of Science and Technology

MASTER IN COMPUTER SCIENCE AND ENGINEERING
SPECIALIZATION IN SPECIALIZATION NAME

NOVA University Lisbon
Draft: *June 28, 2026*

Abstract

Keywords: Multi-Cloud · Generative AI · Cost Optimisation · Cloud Operations · GenAI Frameworks

Resumo

Palavras-chave: Multi-Cloud · Inteligência Artificial Generativa · Otimização de Custos · Operações em Nuvem · Frameworks GenAI

Disclosure of Delegation to Generative Artificial Intelligence

Duarte José Lourenço Belo declares[1] the use of Generative AI in the research work and writing process of this document. According to the GAIDeT taxonomy [35], the following tasks were delegated to Generative AI tools under full human supervision:

Conceptualization

- ☑ Idea generation
- ☑ Defining the research objective
- ☑ Formulating research questions and hypotheses
- ☐ Feasibility assessment and risk evaluation
- ☐ Preliminary hypothesis testing

Literature Review

- ☑ Literature search and systematization
- ☑ Writing the literature review
- ☐ Analysis of market trends and/or patent environment
- ☐ Evaluation of novelty and identification of gaps

Methodology

- ☑ Research design
- ☑ Development of experimental or research protocols
- ☐ Selection of research methods

Software Development and Automation

- ☑ Code generation
- ☑ Code optimization
- ☐ Process automation
- ☐ Creation of algorithms for data analysis

Data Management

- ☑ Data collection
- ☑ Validation
- ☑ Data cleaning
- ☐ Data curation and organization
- ☐ Data analysis
- ☐ Visualization
- ☐ Reproducibility testing

[1] This declaration was generated using LuaLaTeX [36] and the LaTeX package `aidisclose` [26].

Writing and Editing

- [x] Text generation
- [x] Proofreading and editing
- [x] Summarizing text
- [ ] Formulation of conclusions
- [ ] Adapting and adjusting emotional tone
- [ ] Translation
- [ ] Reformatting
- [ ] Press releases and outreach materials

Ethics Review

- [x] Bias analysis and discrimination assessment
- [x] Ethical risk analysis
- [x] Monitoring compliance with ethical standards
- [ ] Data confidentiality monitoring

Supervision

- [ ] Quality assessment
- [ ] Trend identification
- [ ] Identification of limitations
- [ ] Recommendations
- [ ] Publication support

Generative AI tools used: ChatGPT-5.5, Claude 4.6 Sonnet, and Perplexity.

Additional comment #1: The AI was used primarily for refining the code in Section 3, but the algorithm logic was derived manually.

Additional comment #2: This is a second additional comment!

Additional comment: This is a third additional comment! It is starred and, thus, does not get a number. Also, this comment is longer and, hopefully, will span to the next line!

Additional comment #3: This is the third non-starred additional comment!

Contents

List of Figures

List of Tables

LISTINGS

Acronyms

API application programming interface *(pp. 3, 10, 12, 16, 17, 20, 22, 31)*
AWS Amazon Web Services *(pp. 17, 20, 21)*

CapEx capital expenditure *(p. 19)*

DPR Dense Passage Retriever *(pp. vii, 13, 15)*

FinOps cloud financial operations *(pp. 10, 21, 29)*

GCP Google Cloud Platform *(pp. 17, 20, 21)*
GenAI generative artificial intelligence *(pp. vii, viii, 1–4, 6, 7, 9–12, 15, 18, 19, 27–30)*
GRU Gated Recurrent Unit *(p. 9)*

IaC Infrastructure as Code *(p. 20)*

LLM large language model *(pp. vii, 1–3, 6, 7, 9–12, 15, 17–19, 21–32)*
LLMOps large language model operations *(pp. 7, 28, 29)*
LSTM Long Short-Term Memory *(p. 9)*

MCP Model Context Protocol *(pp. vii, 1–3, 5, 9, 17, 32)*
ML machine learning *(p. 27, 28)*

OpEx operational expenditure *(p. 19)*

PII personally identifiable information *(p. 18)*
PPO Proximal Policy Optimization *(p. 12)*

RAG retrieval-augmented generation *(pp. vii, 4, 5, 7, 9, 13–15, 18, 19, 22, 24, 25, 27–30, 35)*
RAGAS Retrieval Augmented Generation Assessment *(pp. 28)*

RAGOps retrieval-augmented generation operations *(pp. 28, 29)*
ReAct reasoning and acting *(pp. vii, 16–18)*
RLHF Reinforcement Learning from Human Feedback *(p. 11, 12)*

SBERT Sentence-BERT *(p. 15)*
SFT supervised fine-tuning *(p. 12)*

1

Introduction

1.1 Context & Motivation

As enterprise adoption of GenAI transitions from exploratory pilots to sustained production use, organisations face a growing set of operational challenges that existing frameworks do not fully address. The rapid proliferation of LLM-based capabilities has exposed critical gaps in how intelligent applications are built, integrated, and governed in production environments: fragmented tool connectivity, limited cost visibility, and insufficient orchestration flexibility [6].

It is precisely in this operational landscape that the present work is situated. The research is grounded in, and developed around, a concrete production-made platform that materialises these requirements in day-to-day production use: Maestro, an enterprise GenAI framework developed at Closer Consulting and deployed in multi-cloud environments. Maestro was conceived to address the operational realities outlined above by providing a structured layer for deploying and managing GenAI workloads across heterogeneous cloud infrastructure. Because it is designed to be instantiated across distinct production environments, the framework must be simultaneously adaptable and robust: it should accommodate diverse model providers, integration patterns, and infrastructure configurations without requiring fundamental redesign at each deployment.

Despite this ambition, three concrete problems currently limit the framework's effectiveness in production. First, tool and service integration remains brittle. In the absence of a standardised connectivity protocol, each new data source or capability requires bespoke integration work, increasing development overhead and reducing portability across deployments [2]. The MCP, introduced by Anthropic in 2024, offers a compelling solution to this problem by defining a universal interface through which LLM hosts can discover and invoke external tools and resources [2]. Integrating MCP into Maestro would enable a plug-and-play connectivity model that scales across production environments without duplicating integration effort.

Second, cost observability is severely constrained. In the current deployment context, LLM infrastructure costs are consolidated under shared cloud resources, with no segmentation by agent, workflow, or request type. This architectural reality makes it impossible to

attribute expenditure to specific system components, which in turn prevents principled optimisation decisions. Without granular cost telemetry in terms of token consumption at the level of individual invocations, teams operating Maestro in production cannot identify inefficient patterns, enforce budget boundaries, or demonstrate the cost-effectiveness of GenAI capabilities to stakeholders [42]. The absence of cost observability is not merely an inconvenience; it is a structural barrier to responsible scaling.

Third, multi-agent orchestration within Maestro lacks the composability required to support complex, interdependent workflows. As GenAI applications grow in scope, they increasingly require the coordination of specialised agents operating in parallel or in structured sequences. Current orchestration approaches do not provide sufficient primitives for expressing these dependencies reliably, which constrains the range of applications that can be built on top of the framework [41].

This dissertation addresses these three problems by proposing a unified and operationally grounded approach. Concretely, the work extends Maestro along three architectural axes: (i) the design and implementation of an MCP-compliant server layer that replaces the current set of bespoke connectors with a standardised, plug-and-play integration model; (ii) the instrumentation of the platform with a unified LLM and infrastructure cost telemetry pipeline, enabling per-invocation token and cost accounting; and (iii) the refactoring of multi-agent workflows around a composable agent-graph orchestration runtime. Together, these extensions are accompanied by a methodology for evaluating production-oriented GenAI infrastructure under operationally realistic conditions.

1.2 Institutional Context

The research presented in this dissertation emerges from a partnership between the Department of Computer Science at NOVA School of Science and Technology (NOVA FCT) and Closer Consulting, which serves as the main industrial collaborator. As the author, I have assumed a dual role - both as an academic researcher and as an active practitioner within Closer Consulting - enabling unique integration of theoretical inquiry and practical implementation.

Closer Consulting is responsible for the design, development, and maintenance of the Maestro GenAI framework, which represents both the technical foundation and the core object of study of this work, functioning simultaneously as the starting point, the engineering substrate on which contributions are built, and the environment in which they are evaluated. Throughout the research process, I have been granted privileged access to Maestro's production deployments, operational cost telemetry, and multi-tenant enterprise configurations, facilitating a level of analysis and validation grounded in production conditions [10].

This collaboration shapes the research agenda directly: architectural decisions, implementation choices, and evaluation metrics are grounded in enterprise constraints leaving the research grounded in production conditions.

1.3 Problem Statement

Despite the growing maturity of enterprise GenAI development, frameworks designed for production use continue to face a set of structural limitations that constrain their scalability, operational efficiency, and portability across deployment environments. This section identifies the three core challenges that motivate the present work, all of which are grounded in the operational reality of Maestro, a framework for enterprise GenAI applications in multi-cloud environments developed at Closer Consulting [10]. Each challenge is mapped explicitly to one or more of the four research objectives stated in Section 1.4: Challenge 1 motivates O1, Challenge 2 motivates jointly O2 and O3, and Challenge 3 motivates O4.

Challenge 1 - Fragmentation of connectivity (motivates O1). The first challenge concerns the standardisation of tool and service integration. Modern enterprise GenAI systems are required not only to consume data from a wide variety of sources - databases, cloud services, analytics platforms, and enterprise APIs - but also to integrate with the capabilities exposed by external technical tools and application programming interfaces (APIs) across heterogeneous infrastructure. In Maestro's current architecture, every connection of this kind is implemented as a bespoke connector, with no shared interface contract governing how tools and resources are discovered or invoked by the underlying LLM [10]. In the absence of such a shared protocol, integration effort is repeated for every new service, the resulting code becomes tightly coupled to the specific provider being consumed, and the system as a whole is difficult to port across deployments without substantial rework - undermining Maestro's multi-cloud design goals. The MCP, introduced by Anthropic in 2024, proposes a universal, open protocol that standardises how LLM hosts discover and invoke external tools and resources [2, 28]. However, the integration of MCP into production-grade GenAI frameworks - including the definition of reusable client and server implementations across cloud platforms - remains an open engineering challenge with limited empirical treatment in the literature [37, 1].

Challenge 2 - Absence of cost observability (motivates O2 and O3). The second challenge concerns cost observability and operational efficiency. In the current deployment context of Maestro, LLM costs are consolidated under a shared cloud bill, with no detail per agent, workflow, or request type. The framework, as deployed today, performs no form of cost or token-consumption analysis: there is no mechanism to determine what costs most, no means of attributing expenditure to specific system components, and consequently no way to justify the investment in GenAI capabilities to stakeholders [42]. Beyond this lack of segmentation, several technical dimensions of cost observability are also absent and are nonetheless essential for the projects built on top of the framework, namely:

1. per-invocation accounting of input, output, and total token counts, together with the resulting monetary cost per model;

2. attribution of those costs to the originating tenant, agent, workflow, prompt template, and RAG pipeline stage;

3. observability of cache behaviour, including semantic-cache hit and miss rates and their effect on token consumption;

4. latency, throughput, and error-rate signals correlated with cost, so that performance and expenditure can be reasoned about jointly; and

5. time-series aggregation that supports budget enforcement, anomaly detection, and longitudinal cost trend analysis.

This absence is not merely a financial inconvenience: it is a structural barrier to making data-driven decisions about which optimisation techniques to apply. Without granular telemetry at the level of individual invocations it is impossible to characterise the production workload - query repetition rate, dominant prompt patterns, average completion length per agent, distribution of model usage - and therefore impossible to select cost-reduction techniques on principled grounds. For this reason, instrumentation (O2) is treated as a hard prerequisite for any cost-reduction work (O3) rather than as a parallel concern. The literature describes a broader set of techniques relevant to O3 - including prompt compression [17, 30], semantic caching [5], dynamic model routing [9], and RAG pipeline optimisation [14] - none of which have been evaluated or implemented within Maestro. A subset of these techniques is later examined in this dissertation; the comparison with prior work is deferred to Chapter **??**, where the approach adopted here is positioned both qualitatively and, where measurements allow, quantitatively against the techniques discussed above. The combined lack of measurement infrastructure and applied optimisation represents a compounded barrier to responsible scaling in production [3].

Challenge 3 - Orchestration without composability (motivates O4). The third challenge concerns orchestration composability for multi-agent workflows. As GenAI applications evolve from single-turn question-answering systems towards complex, multi-step reasoning pipelines, the need for robust multi-agent orchestration primitives becomes increasingly critical [41]. These can vary from simple sequence execution to complex conditional branching and parallel processing. Maestro's agents, however, have been built case by case - for sentiment analysis, session summarisation, relevance checking, and similar tasks - without a shared structural model [10]. Adding new functionality therefore requires modifying existing code rather than composing new behaviour from reusable units, which limits the range of expressible workflows, prevents component reuse, and offers no standard mechanism for inter-agent communication, controlled parallelism, or delegation of subtasks across agents. Equally important, the current implementation provides no execution tracing of what each agent does at runtime: there is no per-step record of inputs, outputs, intermediate decisions, or tool invocations, which makes debugging and quality assurance disproportionately difficult in production and prevents any systematic analysis of where

workflows succeed or fail. As a consequence, the framework cannot readily support the class of complex, interdependent multi-agent applications that are increasingly demanded in enterprise settings [41].

1.4 Objectives

Four research objectives are proposed to address the structural limitations identified in Section 1.3. Each is stated as a concrete, measurable outcome with explicit success criteria and a designated primary toolset. They are not guaranteed to be fully achieved: all four are pursued within the time constraints of the dissertation, and execution follows a fixed order of dependency and priority.

O1 is self-contained and is implemented first. O2 is implemented second and is a hard prerequisite for everything that follows, as it establishes the cost and quality baseline without which O3 and O4 cannot be evaluated. Once O2 is complete, O3 is the primary focus - it addresses the cost-reduction goals central to this dissertation. O4 is only pursued if time remains after O3 is substantially complete, as it represents an independent architectural improvement rather than a dependency of O3.

Although Maestro is designed for multi-cloud deployment, the implementation work undertaken in this dissertation is carried out against a single concrete cloud environment, namely Microsoft Azure, which is the production environment in which the framework is currently operated. Objectives and contributions are therefore framed in terms of the artefacts produced on Azure and the architectural patterns that emerge from them, rather than claims of equivalence across cloud providers. Where validation is still in progress at the time of writing, safegarded language is used to indicate that the stated outcome remains a hypothesis pending empirical confirmation.

O1 - MCP bidirectional integration. The first objective is to introduce, end-to-end, the MCP into Maestro as a first-class connectivity primitive, with the framework acting both as a client and as a server of the protocol [2, 28]. On the client side, Maestro will consume external MCP servers - such as MongoDB, Microsoft Learn, and Fabric MCP - without bespoke per-provider integration code, relying solely on protocol-level discovery and invocation. On the server side, Maestro will expose its own core capabilities - including RAG query, document ingestion, and session summarisation - as protocol-compliant MCP tools that can be invoked by any external MCP-compatible agent. The implementation will use the Anthropic MCP SDK together with FastMCP as the primary tooling, and will be carried out on top of Maestro's current Azure-based production stack. The objective will be considered fulfilled when (i) Maestro successfully consumes at least two heterogeneous external MCP servers without provider-specific integration code, validated by automated integration tests and a dedicated proof-of-concept for each MCP server used; and (ii) the Maestro MCP server exposes at least three core capabilities as valid MCP tools, consumable by at least one independent external client (for example, Claude Desktop or an external LangGraph agent [8]).

O2 - Unified cost telemetry pipeline. The second objective is to instrument Maestro so that, for every LLM invocation, the platform automatically records cost, execution trace, and output quality, thereby producing the first quantitative baseline for the platform. To the best of the author's knowledge, no equivalent cost-observability pipeline currently exists for Maestro, and this baseline is treated as a hard prerequisite for the cost-reduction work undertaken in O3. Langfuse will serve as the LLM observability layer, while Azure Monitor will aggregate infrastructure-level signals via OpenTelemetry, providing a unified operational view across both LLM and compute dimensions on the Azure-hosted deployment of Maestro. Output quality is sampled using the RAGAS evaluation framework [12]. The objective will be considered fulfilled when (i) one hundred per cent of the LLM invocation paths identified in the Maestro code base are instrumented via Langfuse; (ii) each invocation exports its cost per request, its execution trace, and, in sampling mode, its RAGAS quality score; (iii) the costs reported by Langfuse exhibit less than five per cent deviation from the actual Azure billing invoice for the same period, validating the reliability of the baseline; and (iv) the production baseline is available before any O3 intervention begins.

O3 - Cost reduction via tokenomics optimisation. The third objective is to implement and empirically compare at least two complementary cost-reduction techniques, selected on the basis of the patterns identified in the O2 baseline rather than chosen a priori. The selection is therefore data-driven: a workload exhibiting a high query repetition rate motivates semantic caching, whereas a workload dominated by simple queries motivates cascade routing towards smaller models. The candidate techniques are prompt compression via LLMLingua [17], semantic caching via MongoDB Atlas Semantic Cache or Azure Managed Redis (conceptually grounded in GPTCache [5]), and cascade routing via LangChain model routing [8] (inspired by the FrugalGPT cascade strategy [9]). To structure this analysis, a *three-layer tokenomics framework* - presented in detail in Chapter **??** will aim to decompose the cost of a GenAI invocation into the input layer (the tokens sent to the model), the model layer (the cost characteristics of the model that processes the request), and the output layer (the tokens returned to the caller); each candidate technique targets a different layer of this framework. The objective will be considered fulfilled when (i) the selected techniques achieve a significant reduction in cost-per-query relative to the O2 baseline; (ii) quality equivalence relative to the baseline is demonstrated on RAGAS faithfulness and answer relevancy scores via Two One-Sided Tests (TOST) [20] at $\alpha = 0.05$ with an equivalence margin of ± 0.05; and (iii) the evaluation is conducted on real production queries, partitioned into a control group and an experimental group. TOST is adopted in preference to a standard two-sample t-test because the research question is one of equivalence - showing that the experimental and control distributions are sufficiently close to be considered indistinguishable - whereas a non-significant t-test only fails to detect a difference and cannot, on its own, support a positive claim of quality preservation [21].

O4 - Composable agent graph orchestration. The fourth objective is to refactor Maestro's agents as reusable, testable LangGraph [8] graphs, sharing the same observability

stack as O2. LangGraph is selected for three reasons that are architecturally significant. First, Maestro already uses LangChain [8] in production; LangGraph is LangChain's native graph orchestration runtime, making it an extension of the existing stack rather than a replacement, and thereby minimising migration risk. Second, Azure AI Foundry natively hosts LangGraph agents, preserving alignment with the existing Azure infrastructure. Third, Langfuse integrates natively with LangGraph, so that O2 and O4 share a single observability stack - a deliberate architectural decision that eliminates the cost and complexity of maintaining parallel observability pipelines, rather than a limitation imposed by tooling. The objective will be considered fulfilled when (i) at least two production workflows have been refactored as LangGraph graphs with structurally distinct topologies - one sequential and one with conditional branching or parallelism - demonstrating that the LangGraph pattern is expressive enough for diverse real-world cases; (ii) trace coverage of at least eighty per cent of execution steps is achieved via Langfuse; (iii) a reduction of at least thirty per cent in lines of code is achieved relative to the original hand-crafted implementations; and (iv) the refactored agents are accompanied by unit tests, where the original implementations had none.

1.5 Expected Contributions

Pursuing the four research objectives stated in Section 1.4 is expected to produce the following five original contributions:

- **MCP bidirectional integration layer for Maestro.** An MCP client module enabling Maestro to consume external MCP-compliant services without bespoke connectors [2, 28].
- **Unified LLM observability pipeline.** The first integrated observability pipeline for Maestro, covering cost per invocation, multi-step agent execution traces, and output quality scores, implemented via Langfuse and Azure Monitor over OpenTelemetry and validated against production billing data.
- **Empirical benchmarks of cost-reduction techniques.** A statistically rigorous evaluation (TOST, $\alpha = 0.05$) of at least two tokenomics techniques selected on the basis of production usage patterns, conducted on real production queries - providing ecologically valid efficacy estimates that controlled laboratory settings cannot replicate.
- **Quantitative large language model operations (LLMOps) baseline.** A production-derived baseline of token cost, latency, and quality metrics (RAGAS) for a real multi-cloud, multi-tenant RAG system, acting both as a hard prerequisite for O3 and as a reproducible evaluation reference for future comparative studies [12].
- **Reference architecture for composable, observable, and cost-aware GenAI frameworks.** A documented design pattern for enterprise GenAI orchestration in multi-cloud environments, comprising composable LangGraph agent graphs, a shared

Langfuse and OpenTelemetry observability stack, and data-driven cost optimisation - applicable beyond the Maestro platform.

1.6 Structure

2

Background

This chapter provides essential background for the research by introducing the principal concepts and frameworks used in this work. Section 2.1 introduces the foundations of neural language representation. Section 2.2 characterises large language models in production settings. Section 2.3 then covers retrieval-augmented generation, followed by Section 2.4, which addresses LLM orchestration frameworks, the MCP, and the Maestro enterprise GenAI framework. Section 2.5 and Section 2.6 address multi-cloud computing and infrastructure as code, and cost observability and optimization strategies in modern GenAI systems, respectively.

2.1 Foundations of Neural Language Representation

Representing language in a machine-processable form is a prerequisite for any modern GenAI system. Two landmark contributions established the foundations upon which contemporary LLMs and retrieval-augmented architectures are built: dense word representations learned via neural networks, and the Transformer architecture based solely on attention.

2.1.1 Distributed Word Representations

Dense word representations, established by Mikolov et al. [27], demonstrated that semantic similarity can be captured as geometric proximity in a learned continuous vector space - a principle that directly underlies the embedding models and vector databases central to modern RAG pipelines. This representational substrate informs the quality and cost of the embedding stage in the Maestro framework's RAG pipeline, where the choice of embedding model has direct implications for retrieval precision and token expenditure.

2.1.2 The Transformer Architecture

Despite advances in distributed representations, sequence modelling remained constrained by recurrent architectures such as Long Short-Term Memory (LSTM) and Gated Recurrent

Unit (GRU) networks, which process tokens sequentially and suffer from vanishing gradients over long contexts. Vaswani et al. [40] proposed the Transformer, which dispenses with recurrence entirely and relies solely on attention to model dependencies between all positions in a sequence simultaneously. Its core operation, scaled dot-product attention, computes outputs as a weighted sum of value vectors, with weights derived from the compatibility between query and key projections:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

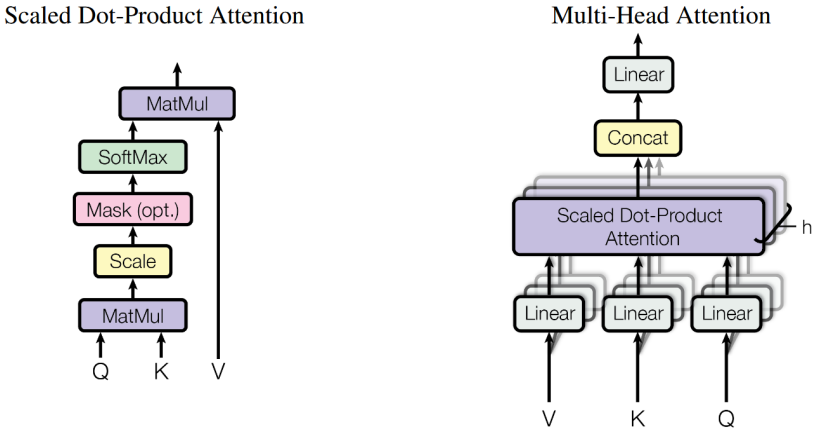

Figure 2.1: Scaled Dot-Product Attention (left) and Multi-Head Attention (right). Adapted from [40].

By running several such operations in parallel - Multi-Head Attention - the model attends simultaneously to different types of relationships, such as syntactic dependencies and semantic co-references. Positional encodings are added to the input embeddings to compensate for the architecture's permutation invariance. The resulting model achieved 28.4 BLEU on the WMT 2014 English-to-German benchmark, surpassing all prior approaches at a fraction of the training compute [40].

The Transformer's parallelism over sequence positions unlocked a new scaling regime. Derived architectures - encoder-only models such as BERT [11] for representation learning, and decoder-only models such as the GPT series [6] for generation - became the standard building blocks of enterprise GenAI systems. Critically, attention scales quadratically with sequence length, $O(n^2)$, making context length a primary cost driver in token-priced LLM APIs. This property is foundational to the cloud financial operations (FinOps) strategies explored in this dissertation - prompt compression, semantic caching, and dynamic model selection - each of which seeks to reduce the effective n processed or the frequency of inference [27, 40].

Together, these works define the representational substrate of modern GenAI: Word2Vec encodes meaning as position in a learned vector space, and the Transformer enables context-sensitive reasoning over such representations efficiently and at scale. The remainder of this

chapter examines the architectural patterns, operational frameworks, and cost optimisation techniques that build upon these foundations in production enterprise systems.

2.2 Large Language Models in Production

Building upon distributed word representations and the Transformer, the field consolidated around a unified paradigm - the LLM - in which ever-larger Transformer networks, pre-trained on internet-scale corpora, acquire general-purpose competence that can be redirected to downstream tasks with minimal additional supervision. This section characterizes LLMs as a class, traces the evolution from raw pre-training to instruction-following alignment, and examines the architectural and economic implications of deploying them at enterprise scale.

2.2.1 Large Language Models: An Overview

A LLM is a neural language model of sufficient parameter count and training data scale that general linguistic and reasoning capabilities emerge from the pre-training objective alone, without task-specific architectural modification [6]. Pre-training proceeds via self-supervised next-token prediction over corpora of hundreds of billions to trillions of tokens, yielding weights that encode an implicit representation of syntax, semantics, world knowledge, and rudimentary reasoning chains, elicited through natural language prompting.

LLMs are predominantly realized as *decoder-only* Transformer architectures [40], whose self-attention layers apply a causal mask restricting each token's attention to its leftward context, preserving autoregressive generation. This contrasts with the encoder-only variant typified by BERT [11], which produces bidirectional representations for discriminative tasks. For generative applications, the decoder-only paradigm dominates, and it is this class of model that underpins the enterprise GenAI systems examined in this dissertation.

A defining empirical finding is the existence of *scaling laws*: power-law relationships between model size, training compute, dataset volume, and downstream performance [6]. Because performance is largely predictable and improvable by increasing any of these axes, compute investment translates into measurable capability gains. This has driven the successive scaling of commercial models and made token consumption a dominant cost variable - larger models process the same input more expensively - directly motivating the cost optimization strategies central to this work.

2.2.2 From Pre-training to Instruction Tuning

Despite their generative competence, early LLMs exhibited systematic misalignment between model behavior and user intent: a model trained only to predict the next token continues text plausibly, but not necessarily in ways that are *helpful*, *honest*, or *harmless* when deployed as an assistant. Ouyang et al. [29] addressed this gap with a three-stage fine-tuning pipeline centered on Reinforcement Learning from Human Feedback (RLHF).

First, a supervised fine-tuning (SFT) phase trains the base model on high-quality human demonstrations over a curated prompt distribution. Second, a *reward model* is trained to predict human preference scores over pairs of outputs, internalizing a proxy for human judgment. Third, the SFT model is optimized against this reward signal using Proximal Policy Optimization (PPO). The resulting InstructGPT models, with as few as 1.3 billion parameters, were consistently preferred by human evaluators over the unaligned 175-billion parameter GPT-3 baseline, showing that alignment quality is not strictly a function of scale [29]. This established RLHF as the standard post-training procedure for production LLMs and confirmed that an aligned smaller model can match or exceed a larger unaligned one - a relationship that informs the dynamic model selection strategies explored in this dissertation.

2.2.3 Open-Weight Models and the Deployment Spectrum

The commercial LLM landscape is stratified between proprietary, API-accessed models and openly released, self-hostable alternatives. The former, exemplified by OpenAI's GPT-4 family and Anthropic's Claude series, exposes inference through token-priced cloud APIs; the latter is represented most prominently by Meta AI's Llama series. Touvron et al. [39] introduced Llama 2, a family of 7 to 70 billion parameter models pre-trained on two trillion tokens and released for commercial use. RLHF-aligned variants (Llama 2-Chat) achieve performance competitive with closed-source alternatives on standard benchmarks while remaining locally deployable [39].

Capable open-weight models introduce a consequential degree of freedom into enterprise GenAI architectures. Rather than committing to a single proprietary provider, organizations may route inference dynamically across a portfolio of models differentiated by capability tier, latency, and per-token cost. Smaller open-weight models, deployed on-premise or on reserved instances, serve high-volume, lower-complexity queries at reduced marginal cost, while frontier API models handle tasks requiring peak reasoning. This routing paradigm - *LLM cascading* or *dynamic model selection* - is a central cost optimization mechanism evaluated in this dissertation in the context of the Maestro framework's multi-cloud deployment [39, 29].

The progression from raw pre-training through RLHF alignment to the open-weight deployment spectrum thus defines the operational environment for enterprise GenAI frameworks. The ability to select, combine, and switch between models of varying capability and cost is not merely an architectural convenience but a financial imperative in production systems, where token consumption can scale three to five times per client without deliberate optimization [29].

2.3 Retrieval-Augmented Generation

Large language models store factual knowledge implicitly within their parameters, which introduces well-documented limitations: parametric knowledge is frozen at training time, cannot be transparently audited, and tends to produce hallucinations when queries exceed the training distribution [24]. RAG addresses these limitations by coupling a generative model with a dynamic, non-parametric memory that can be queried, updated, and inspected at inference time. The following subsections trace the theoretical foundations of the paradigm, its architectural evolution, and the dense retrieval machinery that makes scalable semantic search tractable.

2.3.1 Foundational Architecture

The canonical RAG formulation was introduced by Lewis et al. [24], who proposed a general-purpose fine-tuning recipe for generative models endowed with an external non-parametric memory. The architecture comprises two components: (i) a *retriever* $p_\eta(z \mid x)$, implemented as a DPR [19] whose bi-encoder maps queries and documents into a shared dense vector space via independent BERT encoders, and (ii) a *generator* $p_\theta(y_i \mid x, z, y_{1:i-1})$, implemented as BART-large [23], which conditions its output on the concatenation of the query x and retrieved passages z.

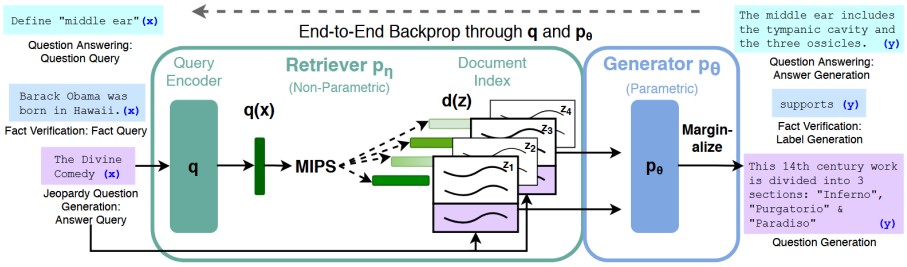

Figure 2.2: End-to-end RAG architecture combining a DPR retriever with a BART-large generator. Adapted from [24].

The retriever and generator are trained end-to-end by minimizing the negative marginal log-likelihood $\sum_j -\log p(y_j \mid x_j)$, without direct supervision on which documents to retrieve. The document encoder and its FAISS index are frozen during fine-tuning; only the query encoder and BART generator are updated, substantially reducing cost.

RAG established state-of-the-art results on open-domain question answering benchmarks, outperforming both parametric seq2seq baselines and task-specific retrieve-and-extract systems [24]. An index-hotswapping experiment further showed that replacing the document index updates the model's world knowledge without retraining, a property of practical significance for production deployments.

2.3.2 Architectural Evolution: Naive, Advanced, and Modular RAG

Gao et al. [14] systematize the rapid evolution of RAG methods into a three-tier taxonomy, providing a structural lens through which the design of contemporary systems can be evaluated.

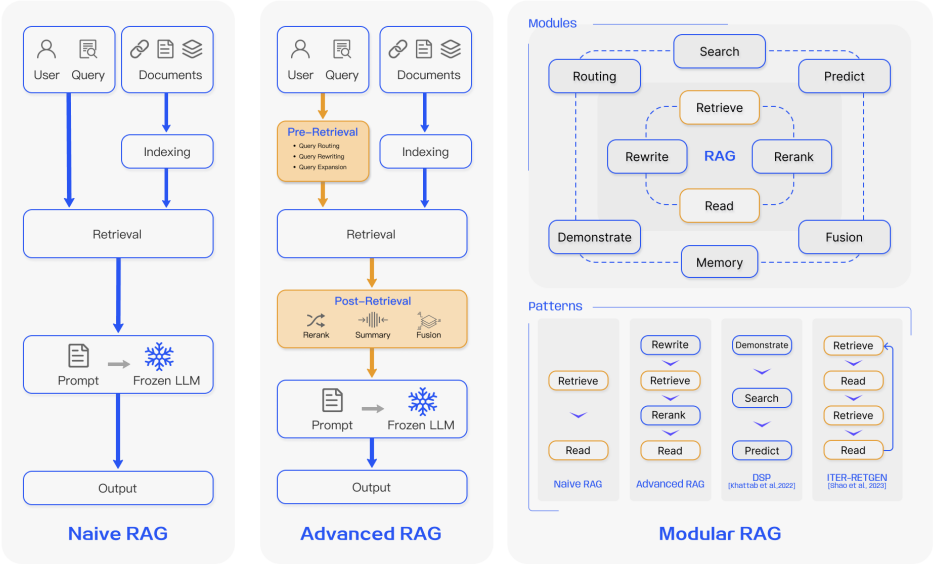

Figure 2.3: Three-tier RAG taxonomy: Naive, Advanced, and Modular RAG. Adapted from [14].

Naive RAG. The earliest implementations follow a minimal *Retrieve-then-Read* pipeline: documents are chunked, encoded, and stored in a vector database; the top-K chunks most similar to the query are retrieved via cosine similarity; and the query and retrieved chunks form a prompt for the language model [14]. While effective in simple scenarios, Naive RAG exhibits well-characterized failure modes: low retrieval precision from semantic gaps, context-window saturation from uninformative passages, and hallucinations when the generator defaults to parametric knowledge amid noisy context.

Advanced RAG. Advanced RAG introduces targeted pre- and post-retrieval interventions to address precision and coherence limitations [14]. Pre-retrieval strategies include query rewriting and expansion into sub-questions; post-retrieval strategies include cross-encoder reranking and context compression to eliminate redundant content before generation. These modifications retain the linear Retrieve-then-Read structure while substantially improving generation quality.

Modular RAG. Modular RAG supersedes the fixed pipeline by decomposing the system into composable functional modules: *Search*, *Memory*, *Routing*, *Predict*, and *Task Adapter* [14].

This supports non-sequential execution patterns, including iterative, adaptive, and self-reflective retrieval patterns [14]. Gao et al. [14] position Modular RAG as the current frontier, whose flexibility allows orchestration patterns that static pipelines cannot accommodate.

2.3.3 Dense Semantic Retrieval and LlamaIndex

Efficient semantic retrieval requires fixed-size sentence embeddings amenable to fast similarity search. Reimers and Gurevych [33] addressed this with Sentence-BERT (SBERT), which modifies BERT using Siamese network structures and mean-pooling to produce embeddings comparable via cosine similarity. By decoupling encoding from comparison, SBERT reduces pairwise sentence comparison from approximately 65 hours to under 5 seconds for a 10,000-sentence corpus, making large-scale semantic search tractable in production RAG systems [33].

LlamaIndex [25] operationalises the full RAG design space surveyed by Gao et al. [14] through a modular architecture separating ingestion, indexing, retrieval, and synthesis. Its `VectorStoreIndex` retrieves nodes via cosine similarity - the same MIPS formulation as DPR [24] - and exposes advanced retrieval patterns including semantic routing, cross-encoder reranking, and iterative retrieval agents. In this work, LlamaIndex serves as the RAG runtime within the Maestro framework, and the choice of embedding model directly determines both retrieval quality and per-query token cost.

2.4 Frameworks, MCP and Maestro

The preceding sections - dense vector representations, attention-based language models, retrieval-augmented generation, and dense semantic retrieval - describe the *components* of a modern GenAI system, but not how they are composed, governed, and operated at production scale. This section addresses that gap across three layers of increasing practical specificity.

2.4.1 LLM Orchestration Frameworks

The emergence of capable LLMs created an immediate engineering challenge: composing LLMs with external tools, persistent memory, and retrieval systems into coherent, repeatable pipelines. LangChain [8] addressed this with three core primitives. A *chain* is a composable sequence of calls - to an LLM, retriever, tool, or another chain - assembled declaratively and reused across applications. A *tool* is any callable capability exposed with a name, a natural-language description, and a typed input schema, which the LLM uses to decide at runtime which tool to invoke and with what arguments. *Memory* modules attach conversational context to a chain, enabling stateful multi-turn interactions. Together these define the dominant orchestration pattern in the field: a model-centric loop in which the LLM selects and sequences external capabilities.

The theoretical foundation for this loop was established by Yao et al. [44], who proposed the ReAct paradigm (*Reason* + *Act*). Rather than separating chain-of-thought reasoning from tool use, ReAct interleaves them in a single iterative cycle: the model emits a *Thought*, selects an *Action* (a tool call), receives an *Observation* (the tool output), and repeats until a terminal condition is met. On multi-step knowledge-intensive tasks - HotpotQA, FEVER, ALFWorld, and WebShop - ReAct outperformed both chain-of-thought and action-only baselines, showing that tight coupling of reasoning and acting is essential for robust agent behavior [44]. This loop is the intellectual ancestor of every subsequent agent executor, including LangChain's `AgentExecutor`, and it directly informs the orchestration design of the system examined in this work.

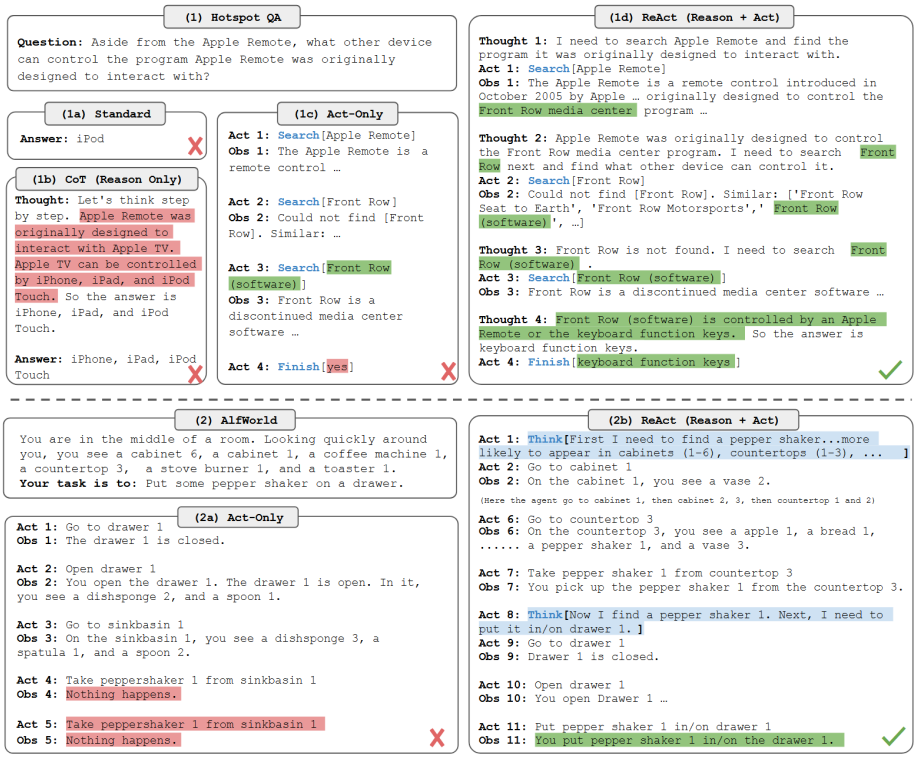

Figure 2.4: The ReAct reasoning loop interleaving *Thought*, *Action*, and *Observation* steps. Adapted from [44].

2.4.2 Model Context Protocol

A persistent weakness of this orchestration paradigm is the absence of a standard interface between agents and the systems they invoke. Each integration is bespoke: a tool written for LangChain is not directly consumable by another framework, and a function registered for OpenAI's function-calling API is invisible to a Claude-based agent. This $N \times M$ integration problem - N agent frameworks times M external services - generates duplicated adapter code, inconsistent error handling, and fragile dependency chains that impede development velocity and maintainability [2].

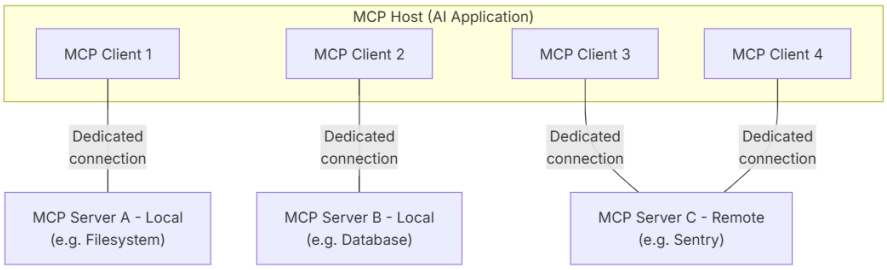

Figure 2.5: MCP architecture showing the Host, Client, and Server roles connected via JSON-RPC 2.0. Adapted from [28].

The MCP, introduced by Anthropic in 2024 [2] and fully specified in its reference documentation [28], addresses this with an open, transport-agnostic protocol standardizing how LLM hosts expose and consume external capabilities. It defines three roles. An *MCP Host* is the application embedding the LLM - a development environment, chat interface, or agent runtime. An *MCP Client* manages the host's connections to servers. An *MCP Server* is a lightweight process exposing a bounded set of capabilities over JSON-RPC 2.0, via either standard I/O (local) or HTTP with Server-Sent Events (remote) [28]. Capabilities are organized into three primitives: *Tools* (functions invoked with structured arguments, analogous to LangChain tools but protocol-defined), *Resources* (read-only data artifacts such as file contents, database schemas, or live API responses), and *Prompts* (parameterized templates offered for reuse) [2, 28].

The distinction from vendor-specific function calling is substantive. OpenAI's mechanism is stateless, scoped to a single provider, and leaves serialization, discovery, and transport to the host. MCP is stateful, provider-agnostic, and supports dynamic capability discovery: a client can query any server for its current tool and resource catalog without prior knowledge of its implementation. This reduces the integration surface from $N \times M$ to $N + M$ - any compliant host interoperates with any compliant server, regardless of the LLM or framework. This interoperability is directly relevant to multi-cloud enterprise deployments, where exposing heterogeneous data services across Azure, Amazon Web Services (AWS), and Google Cloud Platform (GCP) through a single protocol layer has significant governance and maintainability implications.

2.4.3 Enterprise GenAI Frameworks: Maestro in Context

Research prototypes built on LangChain and paradigms such as ReAct operate under assumptions that enterprise production environments do not share. For a deployment serving multiple tenants, correctness and capability are necessary but insufficient: the system must also satisfy identity-based access control, secret management, encryption at rest and in transit, token and model-level cost observability, multi-tenancy isolation, cross-provider portability, and reliability under variable load - none of which are first-class concerns in the orchestration literature above. Meeting these orchestration needs requires

an architectural discipline beyond assembling chains and tools: a governed platform in which each component is independently deployable, swappable without contract changes, and instrumented for both monitoring and audit compliance.

The Maestro GenAI Framework, developed by Closer Consulting, is a practitioner response to precisely this gap [10]. Validated across production deployments in the energy, healthcare, financial, and pharmaceutical sectors, Maestro is a multi-repository, container-native microservices platform that decomposes the RAG pipeline into independently governed services. The *LLM microservice* is a provider-agnostic gateway routing generation requests to Azure OpenAI (GPT-4o, GPT-4o-mini), OpenAI, Azure AI Services (Mistral-Nemo, DeepSeek-R1), and Google Gemini behind a stable contract. The *orchestrator* (chatbot-backend) implements the ReAct-derived RAG loop - history retrieval, hybrid vector search, per-chunk relevance filtering, prompt assembly, and LLM invocation - as a FastAPI service backed by LangChain. The *vector database service* handles ingestion, chunking, embedding via `text-embedding-ada-002`, and hybrid kNN+BM25 retrieval over MongoDB Atlas. The *datastore microservice* provides a provider-agnostic persistence layer over Azure Cosmos DB and MongoDB for chat history, feedback, and document metadata. A *guardrails microservice* performs personally identifiable information (PII) detection via Microsoft Presidio for Portuguese inputs. Two *frontend surfaces* - a Streamlit chatbot with Microsoft Entra ID authentication and an Angular back-office for prompt and question management - complete the user-facing layer. Infrastructure is provisioned declaratively via OpenTofu; although the framework is architected for provider portability, all current production deployments operate on Microsoft Azure, with observability through OpenTelemetry. In production, the framework has demonstrated measurable impact: one deployment serving over 600 daily active users with 4,000 daily interactions across five LLM models, and another achieving 83% question-answering accuracy over 2,907 indexed legal documents [10].

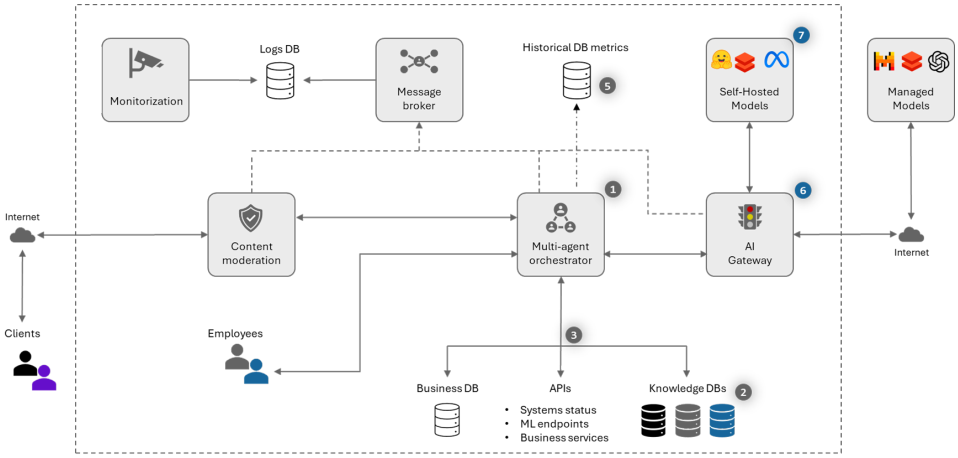

Figure 2.6: Functional architecture of the Maestro GenAI Framework, illustrating the *multi-agent orchestrator*, *AI Gateway*, *content moderation*, self-hosted and managed model endpoints, and observability layer. Adapted from [10].

Despite this maturity, a systematic analysis of the current implementation reveals

structural limitations along three dimensions: the absence of a standardized tool-exposure protocol - each microservice is reachable only through bespoke REST endpoints, with no dynamic capability discovery or external interoperability; an incomplete cost-observability and optimization layer - token usage and per-model cost are not exported as metrics, and no semantic caching or routing reduces redundant invocations; and an orchestration architecture relying on hand-crafted orchestrators rather than a composable, reusable agent graph. These three dimensions constitute the research frontier addressed by the engineering contributions developed in this dissertation.

2.5 Multi-Cloud Computing and Infrastructure as Code

Deploying production-grade GenAI systems at enterprise scale is not a purely algorithmic challenge but equally an infrastructural one. The components surveyed above - LLMs, RAG pipelines, embedding models, and orchestration frameworks - must ultimately be provisioned, networked, secured, and operated on physical or virtual compute. Choices at this layer directly influence cost, resilience, and exposure to vendor lock-in - concerns that Maestro treats as first-class design constraints. This section examines the foundations of cloud computing and multi-cloud strategy, the interoperability and portability challenges of heterogeneous deployments, the research landscape of Infrastructure as Code, and the role of declarative provisioning in making reproducible, provider-agnostic infrastructure management tractable at scale.

2.5.1 Foundations of Cloud Computing

Armbrust et al. [4] identified three properties that distinguish cloud computing from prior infrastructure paradigms: the *illusion of infinite resources* on demand, the *elimination of upfront capital commitments*, and the ability to *pay for resources short-term* and release them when no longer needed. Together these enable a shift from capital expenditure (capital expenditure (CapEx)), where organizations maintain underutilized on-premises hardware, to operational expenditure (operational expenditure (OpEx)), where compute cost tracks actual consumption. Armbrust et al. [4] showed that average server utilization in traditional data centers is only 5–20%, whereas elastic provisioning lets organizations track real demand. This matters for AI-intensive workloads, whose inference load is bursty and hard to predict: on-demand provisioning without upfront commitment is precisely what makes multi-model, multi-provider GenAI deployments economically viable.

The same work identified persistent obstacles to cloud adoption, two of which are relevant here. *Data lock-in* arises when data and application state become tightly coupled to a single provider's proprietary formats, making migration costly and creating asymmetric negotiating power. *Service availability* concerns the concentration of operational risk in one provider, where outages or unilateral policy changes propagate to all dependent workloads. Armbrust et al. [4] identified distributing workloads across independent

providers as the principal mitigation for both. This is the foundational rationale for multi-cloud architectures and anticipates Maestro's design philosophy, which provisions compute and inference capacity across Azure, AWS, and GCP rather than committing to any single platform [10].

2.5.2 Interoperability and Portability in Multi-Cloud Environments

Deploying applications across heterogeneous providers introduces engineering challenges that replication alone does not resolve. Petcu [31] formalized these by distinguishing two concerns: *portability*, the capacity to transfer a workload between cloud environments with bounded effort, and *interoperability*, the capacity for components on different clouds to communicate at runtime. Both require explicit architectural investment and neither emerges spontaneously from layering services across providers.

Petcu [31] identified four sources of lock-in: proprietary APIs exposing non-standard resource abstractions; divergent data formats impeding cross-provider movement; incompatible service-level agreements preventing unified governance; and asymmetric pricing models obscuring total cost of ownership. Each requires a distinct mitigation - respectively abstraction layers, protocol normalization, contractual harmonization, and unified billing instrumentation [31].

Petcu [31]'s taxonomy classifies multi-cloud strategies into three archetypes. A *redundant* strategy designates one provider active and others standby, achieving resilience through replication at the cost of idle resources. A *complementary* strategy assigns distinct capabilities to distinct providers, exploiting each platform's strengths without full duplication. A *federated* strategy presents a unified abstraction to the client, with provider selection transparent to the application [31]. Maestro is designed to operate in complementary mode: AWS Bedrock, Azure OpenAI, and Google Gemini can each be exposed through a uniform `llm-microservice` interface with a stable contract regardless of backend. In current production, inference is routed through Azure-hosted endpoints only; the cross-provider contract nonetheless instantiates the abstraction-layer pattern Petcu [31] advocates for resolving API heterogeneity, and the stable interface is precisely what her interoperability axis prescribes [10].

[TODO] Replace the current AI-generated figure with an original diagram

2.5.3 Infrastructure as Code: Research Landscape and Practice

Infrastructure as Code (Infrastructure as Code (IaC)) is the practice of expressing computing infrastructure configuration as version-controlled, machine-readable source files interpreted by a provisioning engine, rather than manual operator procedures [32]. The declarative approach - where practitioners specify the desired end state and the engine computes convergence - is the dominant paradigm, contrasting with imperative scripting that sequences explicit provisioning steps. Rahman et al. [32] systematically mapped the IaC research landscape, identifying reproducibility, empirical quality assessment, and

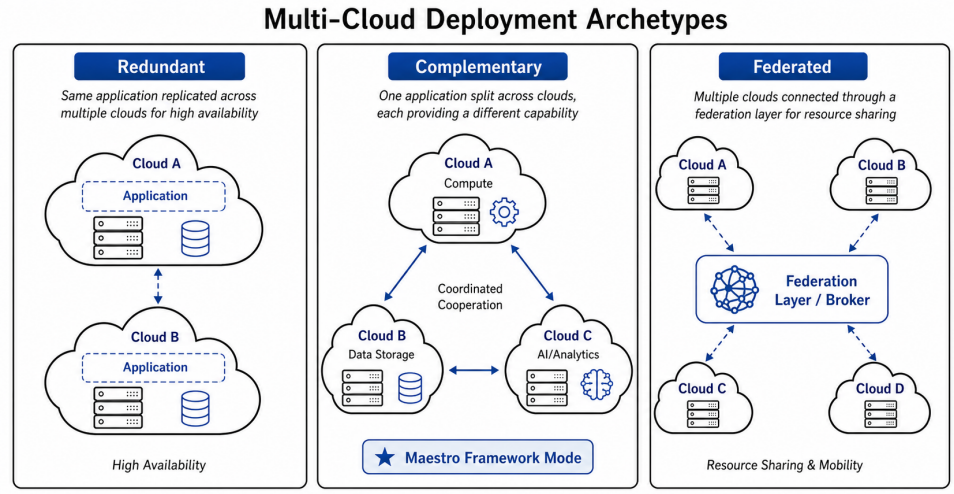

Figure 2.7: Multi-cloud deployment archetypes - redundant, complementary, and federated strategies - as classified by Petcu [31]. The Maestro framework operates in complementary mode.

automated testing as its defining concerns. In the Maestro framework, infrastructure is provisioned via OpenTofu - the Linux Foundation-governed fork of HashiCorp Terraform - with declarative configurations for Azure, AWS, and GCP; current production deployments, however, are limited to Microsoft Azure [10]. Reproducible, version-controlled infrastructure further enables consistent cost attribution across environments: every resource is declared in commit history, allowing cost anomalies to be traced to specific changes and supporting the FinOps disciplines of budget forecasting and spend attribution central to this work [4, 10].

2.6 Cost Optimization

The economic profile of LLM systems is dominated by inference costs that scale with the number of tokens processed, the model's capability tier, and the frequency of invocation. In multi-cloud deployments these costs accumulate across providers and workloads, making cost optimization a first-class architectural concern rather than a post-hoc adjustment [4, 31, 10]. This section surveys four complementary technique classes - semantic caching, model cascades, prompt compression, and prompt caching - and the market context shaping their impact.

All of these mechanisms reduce a system's *effective token cost*. Some reduce the tokens reaching a model (prompt compression, reranking), others reduce the number of model calls (semantic caching, prompt caching), and others reduce the marginal cost per call by routing to cheaper models (LLM cascades). In a multi-cloud setting they compose with provider-specific pricing to determine the realized cost of a workload [14, 10].

A unifying lens is *tokenomics*: minimising token consumption relative to the semantic value delivered per inference call. The token is the atomic economic primitive of LLM

systems - every capability is ultimately paid for in tokens whose price is governed by scaling-law dynamics [6]. Because the cost-accuracy trade-off can be shaped by architectural choice, as FrugalGPT demonstrates by achieving GPT-4-level accuracy at a fraction of its cost via adaptive routing [9], tokenomics is a systems-design discipline, not merely a pricing concern. Market forces alone are insufficient: token prices have fallen substantially [3], yet total inference spend keeps growing as prompts and usage expand. Each technique surveyed here is thus a composable lever over a shared cost function, operating at one of three layers: the *model layer* (cascade routing and dynamic model selection), the *retrieval layer* (reranking and caching), and the *representation layer* (prompt compression, output style, and data serialisation). Advanced RAG pipelines further employ cross-encoder reranking as a post-retrieval precision mechanism; while not a primary implementation target in this work, reranking complements caching and compression by reducing the number of low-relevance tokens forwarded to the generator, thereby operating on the representation layer of the tokenomics framework [14].

2.6.1 Semantic Caching

Semantic caching is motivated by the observation that many production queries are not unique: users repeat, rephrase, or follow up on earlier requests. Traditional caches such as Redis match exact keys and cannot exploit semantic similarity, so even minor paraphrases miss. GPTCache addresses this with an open-source *semantic* cache that stores responses and retrieves them via embedding-based similarity search rather than exact-string matching [5].

Its architecture is straightforward. Incoming queries are encoded - along with stored queries and responses - into embeddings using a model compatible with vector databases such as Milvus or other ANN indexes [5]. A nearest-neighbour search determines whether a semantically similar query was already answered; if similarity exceeds a configurable threshold, the cached answer is returned. Only on a miss is the request forwarded to the LLM, and the new query–response pair inserted for future reuse [5].

Empirically, when integrated with OpenAI's GPT services, cache hits increase response speed by 2–10× while insulating the application from network and provider-side latency spikes [5]. Because an embedding lookup costs negligibly compared to a full LLM call, even a modest hit rate yields tangible API savings - particularly for enterprise assistants, FAQ bots, and RAG systems where queries repeat strongly across tenants and time [5, 10].

In the Maestro framework studied here, semantic caching aligns naturally with existing RAG infrastructure: the system already computes embeddings for retrieval and indexing, so backing a semantic cache requires minimal change. Deployed as a cross-cutting concern in the LLM gateway, it intercepts repeated queries before they reach any provider, reducing both cost and latency [10, 25].

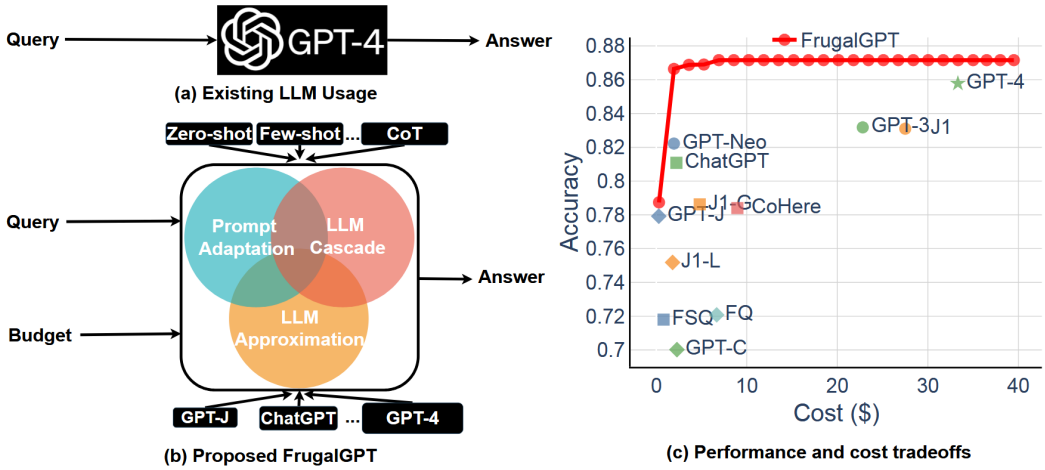

Figure 2.8: Overview of cost-reduction strategies in FrugalGPT, including prompt adaptation, LLM approximation, and LLM cascade. Adapted from Chen et al. [9].

2.6.2 LLM Cascades

Where semantic caching reduces redundant *calls*, LLM cascades target *which* model is invoked when a call is necessary. FrugalGPT studies how to route queries across a portfolio of models to minimize cost subject to accuracy constraints, on the premise that not all requests need the most capable (and expensive) model: many can be handled by smaller models, provided the system can distinguish "easy" from "hard" queries [9].

FrugalGPT proposes several routing strategies: rule-based routing on query features, learned policies predicting which model will answer correctly, and multi-step cascades where a cheap model answers first and escalates only if its answer is judged unsatisfactory [9]. Such cascades reduce inference cost by 50–98% while matching the best single model's accuracy; in some configurations FrugalGPT even exceeds GPT-4 accuracy at equal or lower cost by combining multiple models [9].

This connects to scaling-law and alignment research. Larger models generally perform better but at higher per-token cost [6], while alignment work such as InstructGPT shows smaller, well-aligned models can outperform larger unaligned ones on preference metrics [29]. FrugalGPT leverages both by routing most traffic to small, cheap models and reserving larger models for edge cases that benefit from their capacity [9, 29].

In a provider-portable framework such as Maestro, cascades can be realized at the LLM gateway, which exposes a single logical endpoint while internally routing among on-premise open-weight models (e.g., Llama 2), mid-tier cloud APIs, and frontier proprietary models, per a policy informed by query characteristics, historical performance, and business priorities [10, 39]. This turns model choice from static configuration into a dynamic optimization over latency, accuracy, and cost, tunable per tenant or workload.

2.6.3 Prompt Compression

Prompt compression addresses a different cost driver: the number of tokens processed per inference. Transformer attention scales quadratically with sequence length, so longer prompts cost more and are slower [40, 6]. In enterprise workflows, prompt length is often dominated not by the user's query but by system instructions, conversation history, and retrieved RAG context [14, 10].

Research on prompt compression explores how much input can be removed without materially degrading performance, ranging from heuristic truncation and summarization to learned models that identify salient segments. Jin et al. [18] focus on hardware-level optimizations - storing weights on flash memory and using windowing and row–column bundling to reduce data transfer - but illustrate a broader point: much LLM inference cost stems from unnecessary data movement and redundant processing, at both hardware and token levels [40]. Their cost model accounts for flash characteristics: windowing reuses previously activated neurons to reduce reads, and row–column bundling increases read contiguity. Together these enable running models up to twice the size of available DRAM, with reported 4–5× CPU and 20–25× GPU speed-ups over naive loading [18].

The key analogy for this dissertation is that token-level prompt compression plays the same role at the software layer that windowing and bundling do at the hardware layer: both reduce the information handled per request. In practice, a compression step inserted between retrieval and generation summarizes or filters retrieved passages and history before the LLM call, significantly reducing token usage without sacrificing answer quality, especially in knowledge-intensive tasks with verbose documents [14, 10].

The most academically grounded software-layer approach is the LLMLingua family. Jiang et al. [17] (EMNLP 2023) use a small surrogate model - GPT-2 or LLaMA-7B - to score each token's saliency via conditional perplexity: tokens highly predictable from context carry little unique information and can be removed with minimal impact. Its pipeline has three stages: a budget controller setting the global compression ratio, iterative token-level pruning of low-saliency tokens, and an instruction-tuning alignment stage recovering any quality loss. Empirically, it achieves up to 20× compression while preserving the factual content needed for correct answers [17]. Pan et al. [30] (2024) reformulate compression as token classification: instead of unidirectional perplexity, it trains a bidirectional encoder on data distilled from GPT-4, making retention decisions from both left and right context - task-agnostic, faster at inference, and more faithful than the perplexity-based approach [30]. Both exploit the same gap: a large fraction of tokens in any well-formed prompt are consumed by grammatical predictability - articles, auxiliary verbs, hedging - rather than factual content. Eliminating this overhead removes tokens whose marginal semantic value is near zero, a principle revisited at the representation layer in Section 2.6.6.

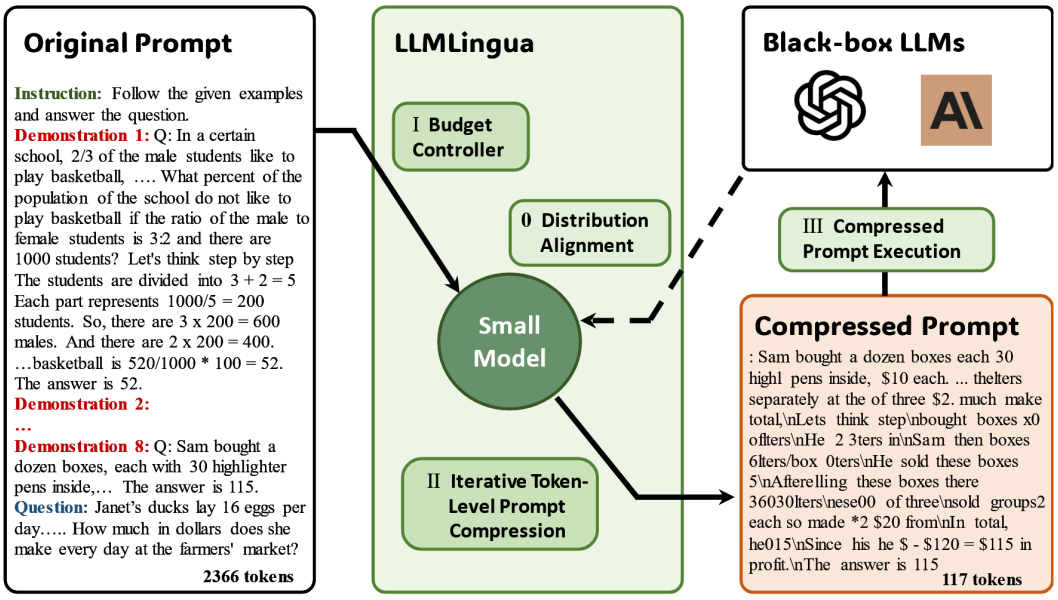

Figure 2.9: The LLMLingua coarse-to-fine prompt compression framework, comprising a budget controller, iterative token-level compression, and a distribution alignment stage. The small surrogate model scores token saliency via conditional perplexity; low-saliency tokens are iteratively removed before the compressed prompt is forwarded to the target LLM. Adapted from Jiang et al. [17].

2.6.4 Prompt Caching

Provider-side prompt caching reuses key-value (KV) cache entries for shared prompt prefixes - long system instructions, policy text, or tool definitions - so that repeated prefixes are billed at a reduced rate rather than recomputed in full. Major proprietary providers now support this mechanism, discounting cached input tokens substantially. Prompt caching is architecturally complementary to semantic caching but operates at the infrastructure layer: it requires exact prefix identity rather than semantic similarity, and it reduces inference cost for repeated prefixes without bypassing the model call itself.

2.6.5 Market Context and LLMFlation

The urgency of cost optimization is shaped by LLM pricing dynamics. Sardana and Frankle [3] introduce "LLMflation" to describe the rapid decline in inference costs, arguing that token prices have dropped by orders of magnitude for comparable capability as infrastructure, hardware, and model efficiency improved; token price indices report similar reductions across proprietary and open-weight ecosystems.

Lower unit prices do not automatically yield lower total spend. As LLMs become cheaper and more capable, organizations expand usage, apply models to more workflows, and supply more context per query, especially in RAG and agentic settings [14, 10]. Demand growth and prompt-length inflation can outpace price cuts, raising absolute cost despite

lower per-token rates - mirroring cloud computing, where falling resource prices often coincided with rising expenditure from workload expansion [4, 31].

The literature therefore argues that cost optimization must be embedded in system architecture rather than treated as a transient concern to be "solved" by future price cuts. Semantic caching, cascades, prompt compression, reranking, and prompt caching remain valuable even as prices fall, because they target structural cost drivers: repeated queries, over-provisioned capacity, unnecessary tokens, and inefficient retrieval [5, 9, 14, 3]. In a multi-cloud environment, these levers also interact with placement and portability decisions that determine how workloads are distributed across providers and pricing regimes [31, 10].

2.6.6 Tokenomics in Practice: Representation and Style as Cost Levers

Beyond model- and retrieval-layer optimisations, token consumption is shaped by choices that precede or follow the model call: the format in which structured data is serialised into the prompt and the verbosity of the requested output. These representation-layer decisions are less studied than caching or routing, but empirical evidence suggests savings comparable to software-layer compression.

TOON (Token-Oriented Object Notation) is a compact drop-in replacement for JSON in LLM prompts that declares column headers once and encodes rows as tab-delimited values, eliminating per-row key repetition [38]. This targets a structural inefficiency inherent to JSON tabular payloads, where schema keys can constitute a significant fraction of prompt length. Practitioner benchmarks report 30–60% token reductions on flat tabular structures without semantic loss [38].

Caveman Compression [7] is a practitioner technique predicated on the observation that LLMs can produce factually equivalent responses in a telegraphic register omitting articles, auxiliary verbs, and hedging. Brussee [7] achieved roughly 75% output token reduction on structured tasks; a subsequent distillation by Guzik [15] reduced the meta-instruction from 552 to 85 tokens while preserving comparable savings. Its mechanistic basis matches the LLMLingua framing of Section 2.6.3: both exploit the gap between tokens consumed by predictable surface forms and those strictly needed for the factual payload, with the distinction that LLMLingua operates on the input while Caveman Compression targets the output. These findings originate in practitioner benchmarks rather than peer-reviewed studies, and are included as empirical evidence motivating formal investigation.

The three-layer tokenomics model can now be stated explicitly. At the *model layer*, cascade routing and dynamic model selection - as formalised by Chen et al. [9] - invoke expensive frontier models only when warranted. At the *retrieval layer*, semantic caching [5], prompt caching, and reranking prevent redundant calls and filter irrelevant context. At the *representation layer*, compression such as LLMLingua [17], format optimisations such as TOON [38], and output style control such as Caveman Compression [7] eliminate tokens whose marginal semantic contribution is low relative to their billing weight. This

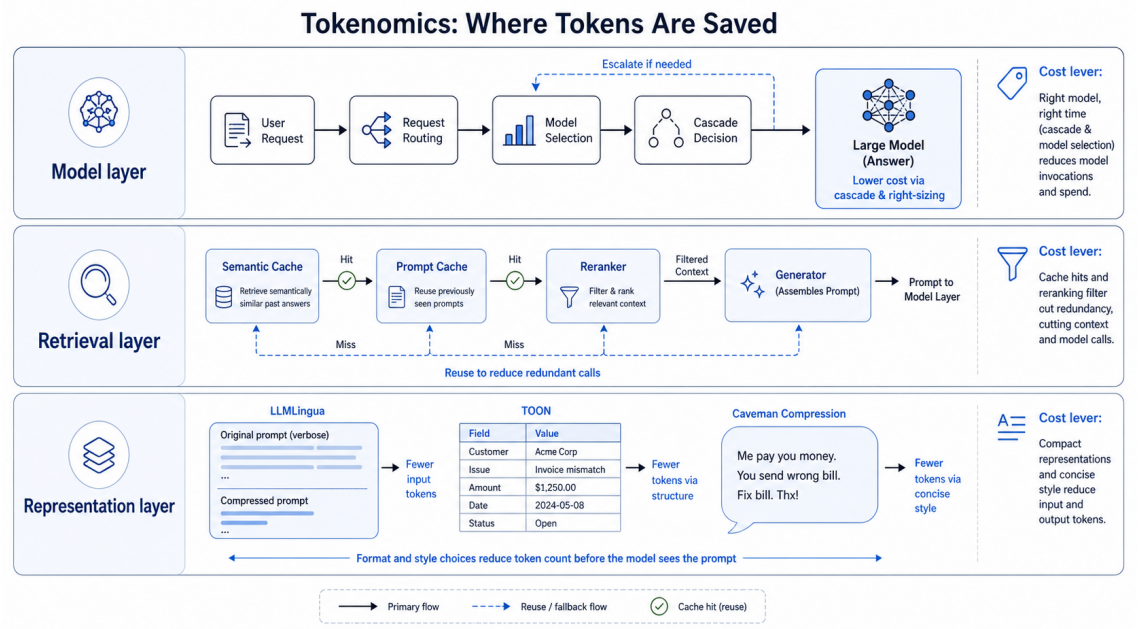

Figure 2.10: Three-layer tokenomics model for cost optimization in enterprise GenAI systems. The model layer controls cascade routing and dynamic model selection; the retrieval layer governs semantic caching, prompt caching, and reranking; the representation layer encompasses software compression (LLMLingua), format optimisation (TOON), and output style control (Caveman). Each layer targets a structurally distinct source of token consumption.

taxonomy frames the engineering contributions of this dissertation, in which the Maestro improvements address cost levers at all three layers within a unified, provider-portable architecture whose current production deployments operate on Azure.

2.7 GenAI Operations and Governance

Deploying Generative AI systems into production introduces operational challenges that extend well beyond traditional software engineering. While earlier sections examined the architectural components of RAG pipelines and the cost dynamics of LLM inference, this section addresses the orthogonal question of how such systems are governed, monitored, and sustained over time. The central argument is that operational governance is not an optional layer atop a working system but a structural requirement whose absence compounds into systemic risk - a perspective grounded in a body of literature that has progressively refined the vocabulary and methodology for managing machine learning (ML) systems in production.

2.7.1 The Technical Debt Perspective on ML Operations

The conceptual foundation for these governance challenges was established by Sculley et al. [34], whose seminal work catalogued the *hidden technical debt* that accumulates in ML

pipelines. ML code typically constitutes a small fraction of the system; the dominant cost lies in surrounding infrastructure - data ingestion, feature engineering, serving, monitoring, and configuration management. This infrastructure is subject to *boundary erosion*, whereby coupling between data distributions, model behaviours, and downstream consumers degrades silently over time, without explicit failures.

Particularly relevant to RAG is the CACE principle - "Change Anything, Changes Everything" - describing how altering a single upstream artefact, such as an embedding model or chunking strategy, may propagate unexpected behaviour through the system [34]. In a production RAG framework such as Maestro this entanglement is amplified: it integrates a continuously evolving knowledge base, a swappable retrieval layer, and a provider-agnostic LLM interface, each a potential source of silent degradation. Sculley et al. [34] thus provide the normative justification for this thesis's governance objectives: without systematic instrumentation, evaluation, and operational discipline, technical-debt accumulation in GenAI systems is not merely possible but inevitable.

2.7.2 Evaluating RAG Pipeline Quality in Production

A RAG governance framework is only as rigorous as its evaluation methodology. Classical metrics such as BLEU and ROUGE are inadequate for retrieval-augmented outputs, evaluating surface-level lexical overlap rather than factual faithfulness or contextual relevance [12]. To address this, Es et al. [12] introduced Retrieval Augmented Generation Assessment (RAGAS) (*Retrieval Augmented Generation Assessment*), a reference-free framework that assesses RAG pipelines along multiple quality dimensions without manually annotated ground truth.

RAGAS decomposes pipeline quality into four metrics: *faithfulness* (how far answers are grounded in retrieved context), *answer relevance* (whether the response addresses the question), *context precision* (whether retrieved passages focus on relevant information), and *context recall* (whether retrieval covers the information needed for a correct answer) [12]. Computed via an LLM-as-judge paradigm, they enable automated, scalable evaluation embeddable directly into continuous integration and monitoring pipelines. RAGAS is the methodological basis for the output-quality objective - O3 in this thesis - and the instrumentation through which Maestro's retrieval and generation quality is tracked in production.

2.7.3 RAGOps: Operationalising the Dual Lifecycle

The rise of RAG as the dominant enterprise LLM paradigm has surfaced operational challenges that existing LLMOps practices cannot address. Xu et al. [43] introduce retrieval-augmented generation operations (RAGOps) to characterise the discipline required to manage RAG pipelines in production, arguing that such systems have a dual lifecycle: an *LLM lifecycle* governing model versioning, prompt management, and inference configuration; and a *data lifecycle* governing the evolution, quality, and freshness of the external

knowledge base.

This framing is consequential for design. In a conventional deployment, the model artefact is the primary unit of management, stable between updates. In a RAG system, by contrast, the knowledge base changes continuously: documents are ingested, revised, or deprecated; embedding distributions shift as indexing evolves; and retrieval relevance may degrade independently of any model change [43]. Xu et al. [43] characterise RAGOps concerns via a 4+1 model view spanning logical, process, development, and physical dimensions. Applied to Maestro, this motivates automated mechanisms for knowledge-base staleness, embedding index versioning, and retrieval-quality monitoring - capabilities not routinely addressed by LLMOps tooling built for static deployments. They additionally report that roughly 60% of enterprise LLM deployments incorporate retrieval augmentation, underscoring the urgency of the discipline.

2.7.4 Financial Governance: The FinOps Lifecycle for AI

Operational governance of GenAI systems also has an economic dimension. Token-priced inference is more dynamic and usage-dependent than the resource-reservation models typical of classical cloud workloads. Without deliberate financial governance, LLM systems are susceptible to overruns from prompt inefficiency, suboptimal routing, or unconstrained retrieval. The FinOps Foundation has extended its cloud financial management framework to these AI dynamics, defining a three-phase cycle: *Inform*, *Optimize*, and *Operate* [13].

The *Inform* phase establishes visibility into AI expenditure by instrumenting token usage, call frequency, and cost per query. The *Optimize* phase uses this visibility to implement improvements - model downsizing for low-complexity queries, semantic caching, prompt compression, and retrieval efficiency. The *Operate* phase embeds these into ongoing practice, establishing cross-functional accountability between engineering, product, and finance and closing the loop iteratively [13]. The cycle is continuous by design: cost governance is positioned not as a one-time exercise but as a persistent discipline integrated into deployment and monitoring.

This informs the cost-efficiency objective - O2 in this thesis - and provides the structure within which the earlier cost-reduction techniques are situated as components of a managed, accountable process rather than isolated interventions. Together, the technical debt perspective, production evaluation methodology, RAGOps lifecycle, and FinOps governance cycle define the operational and financial framework within which the Maestro extensions proposed in this dissertation are situated.

3

System Description and Methodology

This chapter describes the technical platform on which the research contributions are built and the methodology governing their design, implementation, and evaluation. Section 3.1 characterises Maestro - the Closer GenAI Framework - in its pre-intervention state and identifies the structural gaps that motivate each objective. Section 3.2 formalises the research approach and the evaluation instruments applied to each objective.

3.1 System Description

3.1.1 Overview and Architectural Pattern

Maestro is a multi-repository, microservices-based RAG platform for deploying enterprise GenAI applications [10]. Although the LLM gateway is designed to abstract multiple model providers, all production deployments evaluated in this work operate on Microsoft Azure.

Independent FastAPI services communicate over HTTP with API-key authentication in a REST/JSON service mesh, with no message queue or event bus mediating inter-service calls. Service URLs are resolved through Azure App Configuration, secrets through Azure Key Vault, and all Python services share a base Docker image distributed via a shared Azure Container Registry. Infrastructure is provisioned declaratively with OpenTofu, and continuous integration and delivery are handled through Azure DevOps Pipelines.

The framework is implemented in Python 3.12 using FastAPI, LangChain, and Streamlit. As discussed in Section 2.4.1, LangChain provides Maestro's current orchestration primitives, a design decision that directly informs O4. The platform decomposes into four core runtime microservices, three user-facing surfaces, and two platform-layer repositories, summarised in Table 3.1.

This research focuses exclusively on the four core runtime microservices; the user-experience surfaces and the optional guardrails service are not modified and are described only as needed to characterise the system.

Table 3.1: Maestro service tiers and constituent repositories.

Tier	Repositories
Core runtime	`chatbot-backend`
	`llm-microservice`
	`chatbot-vector-db`
	`datastore-microservice`
Optional runtime	`guardrails-microservice`
User experiences	`chatbot-frontend`
	`chatbot-dashboard`
	`backoffice-angular`
Platform	`chatbot-opentofu`
	`demos-framework-root-container`

3.1.2 Core Services

Chatbot Backend (`chatbot-backend`). The backend is the sole orchestrator of the RAG pipeline and the single contact point for all frontend surfaces. It implements the full golden-path sequence - conversation-history retrieval, hybrid vector search, per-chunk relevance filtering, prompt assembly, and LLM invocation - through hand-crafted, non-composable orchestrators, each a standalone script with no shared primitive for branching, parallelism, or execution tracing. As established in Sections 2.3.2 and 2.4.1, this is characteristic of Naive-to-Advanced RAG pipelines that predate composable agent-graph runtimes; O4 addresses it by refactoring the orchestrators as reusable LangGraph graphs [8]. The service also performs no cost or token-consumption instrumentation, the structural gap addressed by O2.

LLM Microservice (`llm-microservice`). This service is a provider-agnostic LLM façade, routing generation requests to Azure OpenAI, OpenAI, Azure AI Services, and Google Gemini behind a stable internal API contract. A routing registry (`REQUEST_PER_SERVICE`) decouples model selection from orchestration logic, enabling model substitution without backend changes. Because queries are dispatched to models of varying capability and cost, the dynamic cascade routing and cost-aware model selection evaluated in O3 are architecturally feasible without redesigning the service [9]. O2 extends this service with Langfuse instrumentation to emit per-invocation token counts and monetary cost for every provider path.

Vector Database Service (`chatbot-vector-db`). This service owns document ingestion and semantic retrieval. It loads documents from Azure Blob Storage, splits them with LangChain's `RecursiveCharacterTextSplitter`, generates dense embeddings via `text-embedding-ada-002`, and stores the vectors in a MongoDB Atlas Vector Search index. At query time it supports both pure k-nearest-neighbour vector search and a hybrid mode combining vector similarity with a BM25-like keyword index, reflecting the Advanced

RAG mechanisms surveyed in Section 2.3.2 [14]. The semantic caching layer evaluated in O3 will be integrated at this retrieval stage, intercepting repeated semantically equivalent queries before the embedding and LLM invocation steps, consistent with the GPTCache architecture of Section 2.6.1 [5].

Datastore Microservice (`datastore-microservice`). This service provides a pluggable transactional data layer for chat history, session metadata, document records, and suggested questions, toggling between Azure Cosmos DB and MongoDB Atlas via a single `DATABASE` environment variable behind a shared API surface. It is not targeted by any of the four objectives but is an operational dependency of the backend orchestrator throughout the evaluation experiments.

3.1.3 Golden-Path Data Flow

The canonical interaction sequence, summarised in Table 3.2, defines the measurement surface for O2 and the optimisation targets for O3.

Table 3.2: Maestro golden-path interaction sequence.

Step	Action
1	The authenticated user submits a query via the Streamlit frontend.
2	The frontend issues `POST /v1/backend/ai-agent-response` to the backend with session, query, and selected provider.
3	The backend retrieves recent conversation history from the datastore.
4	The backend queries the vector database for the top-k similar chunks via hybrid retrieval.
5	The backend evaluates the relevance of each retrieved chunk via the LLM microservice; only passages judged relevant proceed.
6	The backend assembles the prompt and issues a generation call via the selected provider.
7	The answer and source citations are persisted and streamed back to the frontend.

No token counts, latency measurements, or cost signals are recorded at any stage of the current implementation - the structural gap directly motivating O2.

3.1.4 Connectivity and Protocol Gaps

Every external data source or capability in Maestro is currently integrated through a bespoke REST connector, with no shared discovery or invocation contract. The framework has no client layer through which it could consume external MCP-compliant servers without provider-specific adapter code. As discussed in Section 2.4.2, the MCP defines a universal JSON-RPC interface through which LLM hosts discover and invoke external tools and resources, replacing repeated point-to-point integration with a standardised, plug-and-play connectivity model [2]. O1 addresses this gap by introducing an MCP integration

layer that lets Maestro act as a gateway to external MCP servers and additionally exposes Maestro's own core capabilities as protocol-compliant MCP tools.

3.2 Methodology

3.2.1 Research Approach

This work adopts a *Design Science Research* (DSR) methodology [16], the standard paradigm for research whose primary output is an engineered artifact evaluated in a real organisational context. DSR prescribes an iterative cycle of problem identification, artifact design, construction, and rigorous evaluation conducted in close collaboration with the environment that defines the problem. This is appropriate here because the contributions - an MCP integration layer, an observability pipeline, cost-reduction mechanisms, and a composable orchestration runtime - can only be assessed meaningfully under production conditions. The production Maestro deployment on Microsoft Azure serves simultaneously as engineering substrate, experimental environment, and source of evaluation data.

3.2.2 Dependency Ordering and Research Phases

The four objectives are addressed in an order that respects their dependency structure. O1 (MCP integration) is architecturally independent and can proceed in parallel with the instrumentation work of O2. O2 (cost telemetry pipeline) is a *hard prerequisite* for O3: without a quantitative production baseline of token cost, latency, and output quality there is neither a principled basis for selecting cost-reduction techniques nor a reference against which their effect can be measured. O4 (composable agent graphs) reuses the Langfuse instrumentation established in O2 for its trace-coverage evaluation, a deliberate architectural decision that avoids maintaining parallel instrumentation pipelines, as described in Section 2.7 [22].

Each objective follows three phases: *design*, in which the target architecture and tooling are specified; *implementation*, in which the artifact is built and integrated into the Maestro production stack; and *evaluation*, in which the artifact is assessed against the success criteria of Section 1.4 using the measurement instruments described below.

3.2.3 Implementation Approach per Objective

O1 - MCP Bidirectional Integration. Two artifacts are produced: an MCP client that lets Maestro consume external MCP-compliant servers - such as MongoDB Atlas, Microsoft Learn, and Microsoft Fabric MCP - without bespoke adapter code, relying solely on the Anthropic MCP SDK and FastMCP for protocol-level discovery and invocation [2]; and a server interface that exposes Maestro's core capabilities - RAG query, document ingestion, and session summarisation - as protocol-compliant tools consumable by any external MCP-compatible agent.

O2 - Unified Cost Telemetry Pipeline. Every LLM invocation path identified in a static audit of the Maestro codebase is instrumented using Langfuse as the primary LLM observability layer [22]. Each instrumented invocation exports cost per request, a full execution trace, and - in sampling mode - RAGAS quality scores covering faithfulness and answer relevance [12]. Infrastructure-level signals are aggregated through the existing OpenTelemetry pipeline into Azure Monitor, providing a unified view across LLM and compute dimensions. The resulting cost baseline is validated against the corresponding Azure billing invoice before any O3 intervention begins, establishing its credibility as an evaluation reference.

O3 - Cost Reduction via Tokenomics Optimisation. The techniques implemented under O3 are selected from the production usage patterns surfaced by the O2 baseline, following the three-layer tokenomics framework presented in Section 2.6.6. A workload exhibiting high query repetition motivates semantic caching, implemented via MongoDB Atlas Semantic Cache or Azure Managed Redis [5]. A workload dominated by simple queries motivates cascade routing to smaller models via LangChain model routing [9]. Prompt compression via LLMLingua [17] is evaluated as a complementary technique targeting the representation layer. At least two techniques are implemented and empirically compared.

O4 - Composable Agent Graph Orchestration. Maestro's purpose-built orchestrators are refactored as reusable LangGraph graphs, sharing the Langfuse observability stack established in O2 [8]. At least two production workflows with structurally distinct topologies are refactored - one sequential and one with conditional branching or parallelism - demonstrating that the LangGraph pattern is expressive enough for diverse real-world cases. The refactored graphs are accompanied by unit tests, replacing the untested hand-crafted implementations.

3.2.4 Evaluation Strategy

O1. The MCP integration is validated through automated integration tests that verify successful invocation of each exposed MCP tool by an independent external client, with no provider-specific adapter code in the invocation path, and is supported by a proof-of-concept deployment for each integrated external MCP server.

O2. The telemetry pipeline is evaluated on whether the identified LLM invocation paths are instrumented, each invocation exports cost, trace, and sampled RAGAS scores, and the Langfuse-reported costs reconcile with the Azure billing invoice before any O3 intervention begins.

O3. Cost-reduction effectiveness is measured as the percentage reduction in cost-per-query relative to the O2 baseline over a set of production queries partitioned into a control group and an experimental group. Quality preservation is evaluated via the Two One-Sided

Tests (TOST) procedure [20] on RAGAS faithfulness and answer-relevancy scores. TOST is adopted in preference to a standard two-sample t-test because the research question is one of *equivalence* - demonstrating that the experimental and control quality distributions are sufficiently close to be indistinguishable - whereas a non-significant t-test only fails to detect a difference and cannot, on its own, support a positive claim of quality preservation [21].

O4. The orchestration artifacts are evaluated on refactoring at least two workflows with structurally distinct topologies, achieving substantial Langfuse trace coverage of execution steps, reducing lines of code relative to the original hand-crafted implementations, and adding unit tests where the originals had none.

3.2.5 Experimental Controls and Validity Threats

The primary threat to *internal validity* in O3 is the non-stationarity of the production query distribution: if user behaviour shifts between the baseline and intervention period, observed cost differences may reflect workload change rather than technique efficacy. This is mitigated by partitioning queries into control and experimental groups within the same observation window, stratified to preserve query-type distribution.

The primary threat to *construct validity* is the use of Langfuse-reported token counts as a proxy for actual cost; this is addressed by the O2 validation requiring reconciliation with Azure billing data before the instrument serves as an evaluation reference.

External validity is bounded by the single-environment nature of the evaluation: all implementation and measurement work is conducted on the Azure deployment of Maestro. Architectural patterns and cost-reduction findings are expected to transfer to equivalent RAG deployments on other cloud platforms, but cross-environment equivalence is not empirically validated within the scope of the present work.

4

Work Plan

4.1 Research Phases

4.2 Timeline

4.3 Deliverables

Bibliography

[1] Anonymous. "Mitigating Bias and Hallucinations in LLMs through Prompt Engineering and Knowledge-Grounded Approaches in Healthcare Domain: A Systematic Literature Review". In: *IEEE Access* (2025). States explicitly: "MCP emerges as a promising but underexplored tool for dynamic knowledge integration". DOI: 10.1109/ICATC68823.2025.11407841 (cit. on p. 3).

[2] Anthropic. *Model Context Protocol: Specification*. Tech. rep. Anthropic, 2024. URL: https://www.anthropic.com/news/model-context-protocol (cit. on pp. 1, 3, 5, 7, 16, 17, 32, 33).

[3] G. Appenzeller. *Welcome to LLMflation: LLM inference cost is going down fast*. Andreessen Horowitz; accessed May 2026. 2024-11. URL: https://a16z.com/llmflation-llm-inference-cost/ (cit. on pp. 4, 22, 25, 26).

[4] M. Armbrust, A. Fox, R. Griffith, A. D. Joseph, R. H. Katz, A. Konwinski, G. Lee, D. A. Patterson, A. Rabkin, I. Stoica, and M. Zaharia. "A View of Cloud Computing". In: *Communications of the ACM* 53.4 (2010-04), pp. 50–58. DOI: 10.1145/1721654.1721672 (cit. on pp. 19, 21, 26).

[5] F. Bang et al. "GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings". In: *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software* (2023), pp. 212–218. URL: https://aclanthology.org/2023.nlposs-1.24/ (cit. on pp. 4, 6, 22, 26, 32, 34).

[6] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 1877–1901. DOI: 10.48550/arXiv.2005.14165 (cit. on pp. 1, 10, 11, 22–24).

[7] J. Brussee. *Caveman: A Semantic Compression Skill for LLMs*. Claude Code skill; widely replicated; accessed May 2026. 2025. URL: https://github.com/wilpel/caveman-compression (cit. on p. 26).

[8] H. Chase. *LangChain: Building Applications with Large Language Models*. https://langchain.com. Framework documentation and whitepaper. Accessed: 2026. 2022 (cit. on pp. 5–7, 15, 31, 34).

[9] L. Chen, M. Zaharia, and J. Zou. *FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance*. 2023. arXiv: 2305.05176 [cs.CL] (cit. on pp. 4, 6, 22, 23, 26, 31, 34).

[10] Closer Consulting. *Maestro GenAI Framework*. Tech. rep. Internal technical documentation and client deployment case studies. Confidential practitioner source. Closer Consulting, 2025. URL: %5Curl%7Bhttps://closerconsultoria.sharepoint.com/:p:/r/sites/GenAIFramework-DEMO/_layouts/15/Doc.aspx?sourcedoc=%7B42094324-FC65-4A0D-9734-58B50056E83C%7D&file=Closer_Maestro_Framework_Closer_Version.pptx&action=edit&mobileredirect=true%7D (cit. on pp. 2–4, 18, 20–26, 30).

[11] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019-06, pp. 4171–4186. DOI: 10.18653/v1/N19-1423 (cit. on pp. 10, 11).

[12] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". In: *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations (EACL)*. 2024, pp. 150–158. DOI: 10.18653/v1/2024.eacl-demo.16 (cit. on pp. 6, 7, 28, 34).

[13] FinOps Foundation. *FinOps for AI Overview*. Tech. rep. Working group publication. Accessed: May 2026. FinOps Foundation, 2024. URL: https://www.finops.org/wg/finops-for-ai-overview/ (cit. on p. 29).

[14] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023-12. DOI: 10.48550/arXiv.2312.10997 (cit. on pp. 4, 14, 15, 21, 22, 24–26, 32).

[15] K. Guzik. *Caveman-Micro: Distilled Brevity Instruction for LLMs*. Benchmark conducted on Claude Sonnet and Opus; accessed May 2026. 2026. URL: https://github.com/kuba-guzik/caveman-micro (cit. on p. 26).

[16] A. R. Hevner, S. T. March, J. Park, and S. Ram. "Design Science in Information Systems Research". In: *MIS Quarterly* 28.1 (2004), pp. 75–105. DOI: 10.2307/25148625 (cit. on p. 33).

[17] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Yu. "LLMLingua: Compressing Prompts for Accelerated Inference of Large Language Models". In: *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2023. URL: https://arxiv.org/abs/2310.05736 (cit. on pp. 4, 6, 24–26, 34).

[18] X. Jin et al. *LLM in a Flash: Efficient Large Language Model Inference with Limited Memory*. 2023. URL: https://machinelearning.apple.com/research/efficient-large-language (cit. on p. 24).

[19] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih. "Dense Passage Retrieval for Open-Domain Question Answering". In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2020-11, pp. 6769–6781. DOI: 10.18653/v1/2020.emnlp-main.550 (cit. on p. 13).

[20] D. Lakens. "Equivalence Tests: A Practical Primer for t Tests, Correlations, and Meta-Analyses". In: *Social Psychological and Personality Science* 8.4 (2017), pp. 355–362. DOI: 10.1177/1948550617697177 (cit. on pp. 6, 35).

[21] D. Lakens, A. M. Scheel, and P. M. Isager. "Equivalence Testing for Psychological Research: A Tutorial". In: *Advances in Methods and Practices in Psychological Science* 1.2 (2018), pp. 259–269. DOI: 10.1177/2515245918770963 (cit. on pp. 6, 35).

[22] Langfuse. *Langfuse: Open Source LLM Engineering Platform*. Accessed: June 2026. 2024. URL: https://langfuse.com (cit. on pp. 33, 34).

[23] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer. "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension". In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2020-07, pp. 7871–7880. DOI: 10.18653/v1/2020.acl-main.703 (cit. on p. 13).

[24] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 33. 2020-12, pp. 9459–9474. DOI: 10.48550/arXiv.2005.11401 (cit. on pp. 13, 15).

[25] J. Liu. *LlamaIndex*. 2022. URL: https://github.com/jerryjliu/llama_index (cit. on pp. 15, 22).

[26] J. M. Lourenço. *The `aidisclose` package: Generative AI disclosure checklist and statements*. Version 1.6.2. 2025. URL: https://ctan.org/pkg/aidisclose (cit. on p. iii).

[27] T. Mikolov, K. Chen, G. Corrado, and J. Dean. *Efficient Estimation of Word Representations in Vector Space*. 2013-09. DOI: 10.48550/arXiv.1301.3781 (cit. on pp. 9, 10).

[28] Model Context Protocol Contributors. *Model Context Protocol — Reference Documentation*. https://modelcontextprotocol.io. Canonical technical specification. Accessed: 2026. 2024 (cit. on pp. 3, 5, 7, 17).

[29] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, P. Mishkin, C. Zhang, S. Agarwal, K. Slama, A. Ray, J. Schulman, J. Hilton, F. Kelton, L. Miller, M. Simens, A. Askell, P. Welinder, P. Christiano, J. Leike, and R. Lowe. "Training Language Models to Follow Instructions with Human Feedback". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 35. 2022-12, pp. 27730–27744. DOI: 10.48550/arXiv.2203.02155 (cit. on pp. 11, 12, 23).

[30] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, J. Zhang, Q. Lin, V. Rühle, Y. Yang, C.-Y. Lin, H. V. Zhao, L. Qiu, and D. Zhang. "LLMLingua-2: Data Distillation for Efficient and Faithful Task-Agnostic Prompt Compression". In: *arXiv preprint arXiv:2403.12968* (2024). URL: https://arxiv.org/abs/2403.12968 (cit. on pp. 4, 24).

[31] D. Petcu. "Portability and Interoperability between Clouds: Challenges and Case Study". In: *Towards a Service-Based Internet*. Lecture Notes in Computer Science 8135 (2013), pp. 62–74. DOI: 10.1007/978-3-642-45015-0_6 (cit. on pp. 20, 21, 26).

[32] A. Rahman, R. Mahdavi-Hezaveh, and L. Williams. "A Systematic Mapping Study of Infrastructure as Code Research". In: *Information and Software Technology* 108 (2019-04), pp. 65–77. DOI: 10.1016/j.infsof.2018.12.004 (cit. on p. 20).

[33] N. Reimers and I. Gurevych. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 2019-11, pp. 3982–3992. DOI: 10.18653/v1/D19-1410 (cit. on p. 15).

[34] D. Sculley, G. Holt, D. Golovin, E. Davydov, T. Phillips, D. Ebner, V. Chaudhary, M. Young, J.-F. Crespo, and D. Dennison. "Hidden Technical Debt in Machine Learning Systems". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 28. 2015. URL: https://papers.neurips.cc/paper/5656-hidden-technical-debt-in-machine-learning-systems.pdf (cit. on pp. 27, 28).

[35] Y. Suchikova, N. Tsybuliak, J. A. T. da Silva, and S. Nazarovets. "GAIDeT (Generative AI Delegation Taxonomy): A Taxonomy for Humans to Delegate Tasks to Generative Artificial Intelligence in Scientific Research and Publishing". In: *Accountability in Research* (2025), pp. 1–27. DOI: 10.1080/08989621.2025.2544331 (cit. on p. iii).

[36] The LaTeX Project team. *LaTeX - A document preparation system*. 1985. URL: https://www.latex-project.org (visited on 2025-12-29) (cit. on p. iii).

[37] Thoughtworks. *Technology Radar Volume 33*. Technical Report. Places MCP in "Trial"; identifies rapid industrial adoption of MCP-powered agents as a defining theme of 2025. Thoughtworks, 2025-11. URL: https://www.thoughtworks.com/radar (cit. on p. 3).

[38] *TOON: Token-Oriented Object Notation*. Practitioner serialization format for LLM prompt token efficiency; accessed May 2026. 2025. URL: https://openapi.com/blog/what-the-toon-format-is-token-oriented-object-notation (cit. on p. 26).

[39] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale, D. Bikel, L. Blecher, C. C. Ferrer, M. Chen, G. Cucurull, D. Esiobu, J. Fernandes, J. Fu, W. Fu, B. Fuller, C. Gao, V. Goswami, N. Goyal, A. Hartshorn, S. Hosseini, R. Hou, H. Inan, M. Kardas, V. Kerkez, M. Khabsa, I. Kloumann, A. Korenev, P. S. Koura, M.-A. Lachaux, T. Lavril, J. Lee, D. Liskovich, Y. Lu, Y. Mao, X. Martinet, T. Mihaylov, P. Mishra, I. Molybog, Y. Nie, A. Poulton, J. Reizenstein, R. Rungta, K. Saladi, A. Schelten, R. Silva, E. M. Smith, R. Subramanian, X. E. Tan, B. Tang, R. Taylor, A. Williams, J. X. Kuan, P. Xu, Z. Yan, I. Zarov, Y. Zhang, A. Fan, M. Kambadur, S. Narang, A. Rodriguez, R. Stojnic, S. Edunov, and T. Scialom. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. 2023-07. DOI: 10.48550/arXiv.2307.09288 (cit. on pp. 12, 23).

[40] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin. "Attention Is All You Need". In: *Advances in Neural Information Processing Systems (NeurIPS)*. Vol. 30. 2017-12. DOI: 10.48550/arXiv.1706.03762 (cit. on pp. 10, 11, 24).

[41] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen. *A Survey on Large Language Model Based Autonomous Agents*. 2023. DOI: 10.48550/arXiv.2308.11432 (cit. on pp. 2, 4, 5).

[42] C.-J. Wu, R. Raghavendra, U. Gupta, B. Acun, N. Ardalani, K. Maeng, G. Chang, F. A. Behram, J. Huang, C. Bai, M. Gschwind, A. Gupta, M. Ott, A. Melnikov, S. Candido, D. Brooks, G. Chauhan, B. Lee, H.-H. S. Lee, B. Agrawal, M. Fixler, S. Sheahan, S. Sridharan, A. M. Zain, and M. Smelyanskiy. "Sustainable AI: Environmental Implications, Challenges, and Opportunities". In: *Proceedings of Machine Learning and Systems (MLSys)*. Vol. 4. 2022, pp. 795–813. URL: https://arxiv.org/abs/2111.00364 (cit. on pp. 2, 3).

[43] X. Xu et al. *Operating and Managing Retrieval-Augmented Generation Pipelines*. 2025. arXiv: 2506.03401 [cs.SE] (cit. on pp. 28, 29).

[44] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *Proceedings of the Eleventh International Conference on Learning Representations (ICLR)*. 2023. URL: https://arxiv.org/abs/2210.03629 (cit. on p. 16).

A

Appendix Title